ОСНОВЫ ОБЛАЧНЫХ ТЕХНОЛОГИЙ

Занятие 5
КОНТЕЙНЕРИЗАЦИЯ ПРИЛОЖЕНИЙ. DOCKER

Теоретические сведения, разобранные примеры,
контрольные вопросы и лабораторная работа № 5

Рабочая среда. Все задания выполняются на собственном компьютере под управлением Windows, macOS или Linux, без учётной записи AWS и без каких-либо затрат. Программы на компилируемых языках собираются внутри контейнеров Docker, поэтому команды одинаковы во всех операционных системах. В Windows команды вводятся в терминале WSL (Ubuntu).

Язык реализации. Опорные тексты курса написаны на C++, а сервис модели и конвейеры обработки данных — на Python. Собственные компоненты допускается выполнять на другом языке программирования при четырёх условиях: сборка или запуск возможны внутри контейнера Linux; для языка есть библиотека HTTP-сервера, а для занятий 7 и 9 — поддержка Protocol Buffers и gRPC; доступна 64-разрядная целочисленная арифметика, нужная генератору котировок; язык согласован с преподавателем и указан в отчёте. Выбранный язык не меняется до конца курса: сквозной проект собирается из компонентов всех занятий. Требования, которые от языка не зависят, — контракт интерфейса, значения индикаторов, свойства образа, оформление схем и расчётов — перечислены в критериях приёма; формулы и контрольные значения приведены в приложении Г к документу занятия 4 «Опорная библиотека: генератор котировок и индикаторы». Занятия 1, 2 и 13 выполняются на C++ независимо от сделанного выбора: их предмет — сама система сборки CMake и различия в поведении одной и той же ошибки при разных режимах сборки.

Содержание

1. Цель занятия и требования к подготовке

2. Рабочая среда: установка Docker

- 2.1. Windows
- 2.2. macOS
- 2.3. Linux
- 2.4. Проверка

3. Что такое контейнер

- 3.1. Контейнер и виртуальная машина
- 3.2. Пространства имён
- 3.3. Контрольные группы
- 3.4. Слоистая файловая система
- 3.5. Стандарт OCI и среды выполнения

4. Образы и файл Dockerfile

- 4.1. Инструкции
- 4.2. Слои и кэш сборки
- 4.3. Контекст сборки
- 4.4. ENTRYPOINT и CMD
- 4.5. Многоархитектурные образы
- 4.6. Возможности современного сборщика

5. Многоэтапная сборка и минимальные образы

- 5.1. Происхождение образа и его проверка

6. Запуск контейнеров

- 6.1. Основные параметры
- 6.2. Тома и монтирование
- 6.3. Ограничение ресурсов
- 6.4. Диагностика
- 6.5. Процесс номер один: сигналы и завершение
- 6.6. Ограничение процессора: доли, квоты и торможение

7. Amazon ECR: реестр образов

- 7.1. Имя образа и дайджест
- 7.2. Реестр рядом с нагрузкой
- 7.3. Сканирование образов
- 7.4. Политика хранения

8. Amazon ECS: оркестрация контейнеров в облаке

- 8.1. Определение задачи, задача и служба
- 8.2. Запуск без управления серверами
- 8.3. Узлы или без узлов: как выбирать
- 8.4. Другие способы запустить образ

9. Эксплуатация контейнеров в облаке

- 9.1. Две роли
- 9.2. Журналы
- 9.3. Секреты
- 9.4. Проверка состояния и обновление службы
- 9.5. Хранилище для задач
- 9.6. Что измерять у контейнера

10. Эталонная архитектура контейнерного приложения

11. Способы запуска контейнера в облаке

- 11.1. Ограничения, определяющие выбор
- 11.2. Разобранный пример выбора

12. Стоимость контейнерной нагрузки

- 12.1. Расчёт для опорной композиции
- 12.2. Способы снижения стоимости

13. Разобранные примеры

14. Задачи для самопроверки

15. Типичные ошибки и контрольные вопросы

- 15.3. Ошибки, которые проявляются только в облаке

16. Лабораторная работа № 5

- 16.1. Цель и порядок
- 16.2. Отчёт
- 16.3. Варианты
- 16.4. Задания 1—6
- 16.5. Задания 7—9
- 16.6. Задания повышенной сложности
- 16.7. Критерии
- 16.8. Этап сквозного проекта

17. Подготовка к итоговой аттестации

18. Приложения

1. Цель занятия и требования к подготовке

Цель занятия

Научиться упаковывать приложение вместе с его окружением в контейнер, управлять размером и безопасностью образа и понимать, как тот же образ запускается в облаке без изменений.

Задачи занятия

1. Разобрать механизмы ядра, на которых построены контейнеры, и отличие контейнера от виртуальной машины.
2. Освоить написание Dockerfile, устройство слоёв и работу кэша сборки.
3. Применить многоэтапную сборку и сравнить размеры образов.
4. Научиться запускать контейнеры с ограничением ресурсов, томами и непривилегированным пользователем.
5. Соотнести локальный запуск с сервисами контейнеров AWS: ECR, ECS, Fargate.

Требования к предварительной подготовке

— сборка проекта CMake, статические и динамические библиотеки (занятия 1 и 2);

— понятие процесса, пользователя и прав доступа в Linux;

— виртуализация и гипервизоры (занятие 4): Docker Desktop использует виртуальную машину Linux;

— работа в командной строке.

Ожидаемые результаты

— объяснять, что изолирует контейнер и чем он отличается от виртуальной машины;

— писать Dockerfile с многоэтапной сборкой и непривилегированным пользователем;

— управлять кэшем сборки осознанно, а не перебором;

— запускать контейнеры с пределами ресурсов и сохранять результаты работы вне контейнера;

— соотносить локальные действия с сервисами ECR, ECS и Fargate.

Вклад в сквозной проект. Библиотека quant-core впервые попадает в контейнер: многоэтапный образ, непривилегированный пользователь, проверка состояния и собственный индикатор варианта. Сохраните: Dockerfile, имя и тег образа, файл схемы ECR → ECS Fargate. Опирается на занятия 1, 2, 4. Понадобится на занятиях 8, 10, 13 и 15: этот образ разворачивается в композиции, в конвейере и в кластере.

2. Рабочая среда: установка Docker

Docker состоит из служебного процесса (демона), который управляет образами и контейнерами, и клиента командной строки `docker`. В Windows и macOS ядра Linux нет, поэтому Docker Desktop запускает виртуальную машину Linux и выполняет контейнеры в ней — это прямое применение материала занятия 4.

> **Условия использования.** Docker Desktop бесплатен для личного использования, образования и некоммерческих проектов, а также для организаций численностью менее 250 сотрудников с годовой выручкой менее 10 млн долларов. Условия приведены в документации Docker.

2.1. Windows

1. Включите аппаратную виртуализацию в прошивке (проверка — занятие 4) и компонент WSL версии 2.
2. Установите Ubuntu из Microsoft Store: `wsl --install -d Ubuntu`.
3. Установите Docker Desktop с сайта docker.com; в параметрах включите интеграцию с дистрибутивом WSL (Settings → Resources → WSL integration).
4. Все команды занятия выполняются в терминале Ubuntu (WSL), а не в PowerShell: так они совпадают с командами в macOS и Linux.

2.2. macOS

Установите Docker Desktop для macOS, выбрав сборку для своего процессора (Apple Silicon или Intel). После установки запустите приложение: оно поднимает виртуальную машину Linux.

2.3. Linux

Установите Docker Engine из репозитория дистрибутива или по инструкции docs.docker.com. В Ubuntu:

```
sudo apt-get update && sudo apt-get install -y docker.io docker-compose-v2
sudo usermod -aG docker $USER      # чтобы не писать sudo перед каждой командой
newgrp docker                      # применить группу в текущей сессии
```

> **Важно.** Членство в группе docker равносильно правам суперпользователя на узле: процесс в контейнере можно запустить от root и смонтировать любой каталог хоста. На общих серверах этим правом не злоупотребляют, а в промышленной эксплуатации применяют режим без прав суперпользователя (rootless).

2.4. Проверка

```
docker version            # версии клиента и сервера
docker info | head -20    # сведения о системе: ОС, архитектура, драйвер хранилища
docker run --rm hello-world
```

В поле `Operating System` вывода `docker info` в Windows и macOS указана Linux-система виртуальной машины Docker Desktop, а не основная ОС. Поле `Architecture` показывает архитектуру: `aarch64` на компьютерах Apple Silicon, `x86_64` на остальных. Это важно: образы собираются под архитектуру, и образ, собранный на одной архитектуре, не запустится на другой без эмуляции.

3. Что такое контейнер

3.1. Контейнер и виртуальная машина

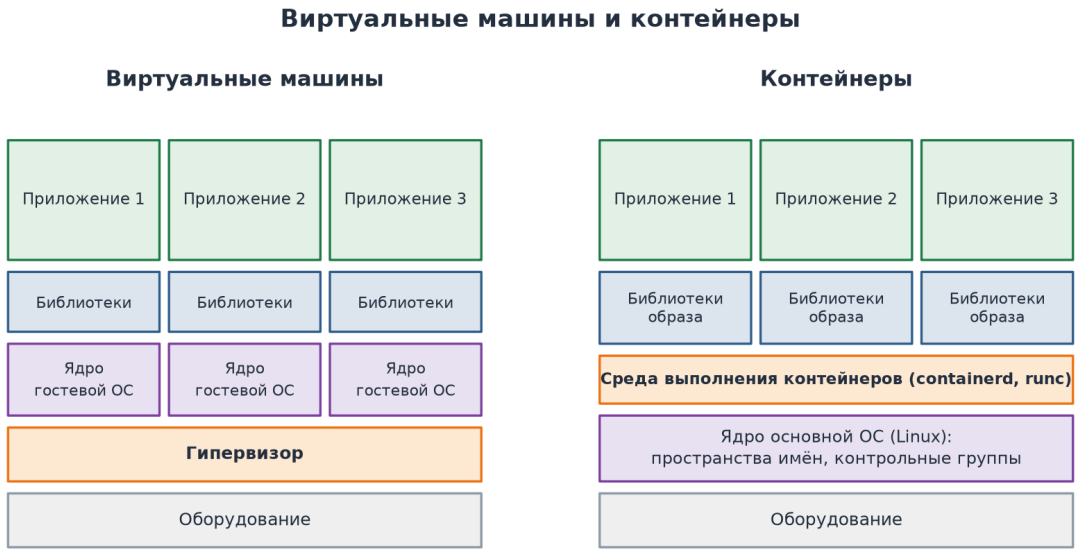


Рисунок 3.1 — Виртуальные машины и контейнеры

Контейнер — это обычный процесс основной операционной системы, которому ядро подменило представление о системе: он видит своё дерево файлов, свои процессы, свою сеть. Никакого «облегчённого ядра» внутри контейнера нет. Поэтому контейнер запускается за доли секунды, а его образ содержит только пользовательскую часть системы.

3.2. Пространства имён

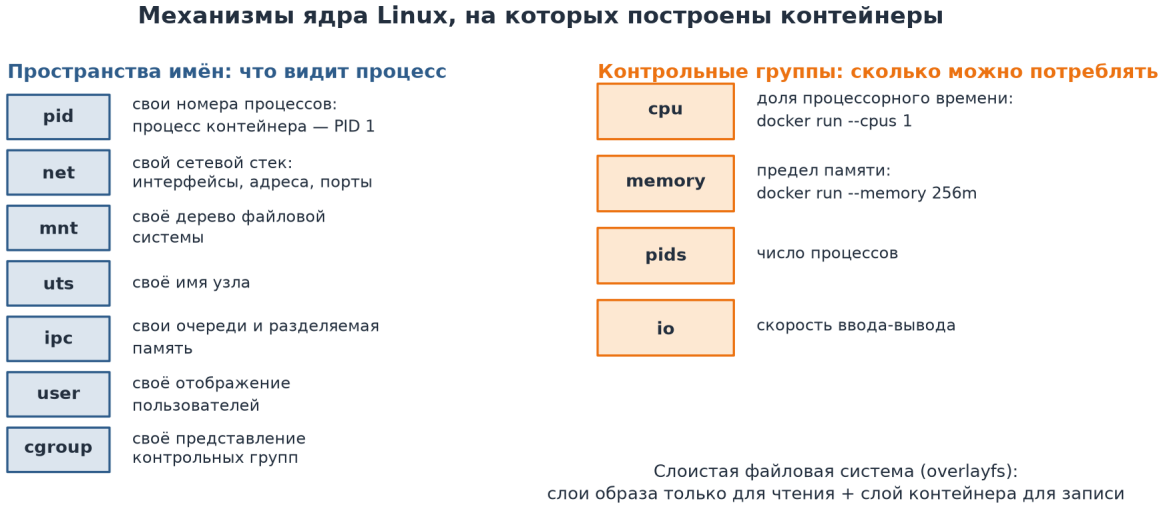


Рисунок 3.2 — Механизмы ядра Linux

Пространства имён (namespaces) определяют, *что* видит процесс. Для каждого вида ресурса ядро ведёт отдельное пространство: процессы, сеть, точки монтирования, имя узла, средства межпроцессного взаимодействия, пользователи. Процесс, помещённый в новое пространство имён процессов, получает номер 1 и не видит процессов хоста.

3.3. Контрольные группы

Контрольные группы (cgroups) определяют, *сколько* процесс может потребить: доля процессорного времени, предел памяти, число процессов, пропускная способность ввода-вывода. Пределы, заданные параметрами `--cpus` и `--memory`, видны внутри контейнера в файловой системе cgroup версии 2.

```
docker run --rm --cpus 1 --memory 256m --entrypoint sh quant-cli:multi \
  -c 'cat /sys/fs/cgroup/cpu.max /sys/fs/cgroup/memory.max'
100000 100000        # 100 000 мкс на 100 000 мкс периода = одно ядро
268435456            # 256 МиБ
```

3.4. Слоистая файловая система

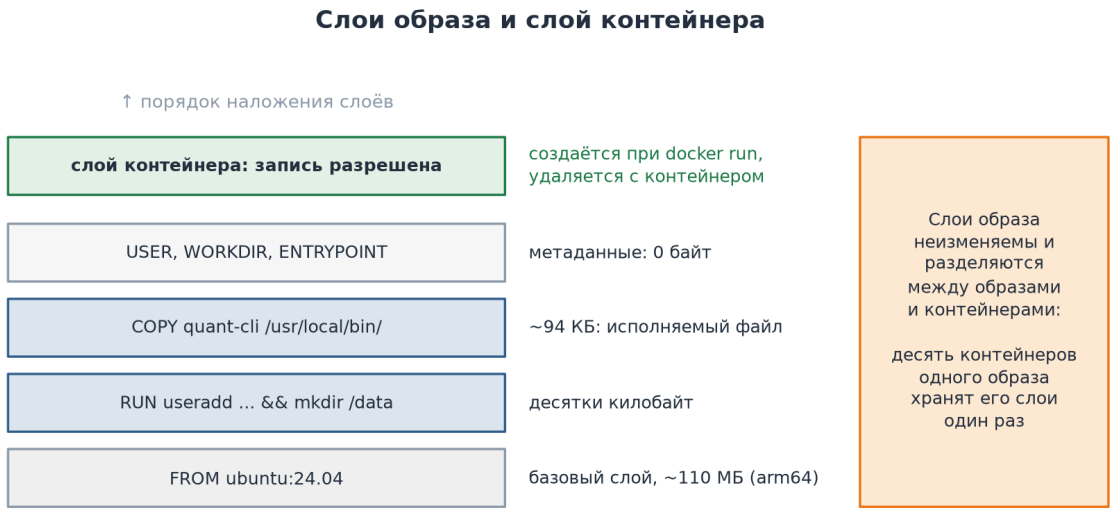


Рисунок 3.3 — Слои образа и слой контейнера

Файловая система контейнера собирается из слоёв: слои образа доступны только для чтения, сверху добавляется слой контейнера для записи (механизм overlayfs). Изменённый файл копируется в верхний слой (копирование при записи), поэтому запись не портит образ. Слои разделяются между всеми контейнерами и образами: десять контейнеров одного образа хранят его слои один раз.

3.5. Стандарт OCI и среды выполнения

Формат образа и правила его запуска стандартизованы Open Container Initiative (OCI). Поэтому образ, собранный Docker, запускается в Kubernetes, в AWS Fargate или инструментом Podman. Docker использует среду выполнения containerd, которая вызывает низкоуровневую среду runc, непосредственно создающую пространства имён и контрольные группы.

4. Образы и файл Dockerfile

4.1. Инструкции

Инструкция	Назначение	Создаёт слой
FROM	базовый образ; начинает этап сборки	да (слои базового образа)
RUN	выполнить команду при сборке	да
COPY	скопировать файлы из контекста сборки или из другого этапа	да
WORKDIR	рабочий каталог для последующих инструкций	нет (метаданные)
ENV, ARG	переменные окружения образа и аргументы сборки	нет
USER	пользователь, от имени которого работает процесс	нет
EXPOSE	документирование порта (порт не публикуется)	нет
VOLUME	точка монтирования данных	нет
HEALTHCHECK	команда проверки состояния	нет
ENTRYPOINT, CMD	что выполняется при запуске контейнера	нет

4.2. Слои и кэш сборки

Сборщик выполняет инструкции по порядку и кэширует результат каждой. Кэш используется, пока совпадают сама инструкция и её входные данные: для `RUN` — текст команды, для `COPY` — содержимое копируемых файлов. Как только одна инструкция изменилась, все последующие выполняются заново. Отсюда правило: **редко меняющееся — выше, часто меняющееся — ниже**.

Порядок инструкций и кэш

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y --no-install-recommends build-essential cmake \
    && rm -rf /var/lib/apt/lists/*      # редко меняется: остаётся в кэше
WORKDIR /src
COPY . .                                # часто меняется
RUN cmake -S . -B build && cmake --build build && ctest --test-dir build
```

Обратите внимание. Установка пакетов и удаление списка пакетов выполняются **одной** инструкцией RUN. Если удалить файлы отдельной инструкцией, они останутся в предыдущем слое и образ не уменьшится: слои складываются, а не заменяются.

4.3. Контекст сборки

Команда `docker build <каталог>` передаёт сборщику содержимое указанного каталога — **контекст сборки**. Инструкция COPY может копировать только из него, поэтому путь вида `../file` недопустим. Файл `.dockerignore` исключает из контекста ненужное: каталоги сборки, служебные каталоги систем контроля версий, файлы с секретами.

.dockerignore проекта

```
build/
.git/
**/__pycache__/
*.drawio
```

4.4. ENTRYPOINT и CMD

`ENTRYPOINT` задаёт исполняемый файл, `CMD` — аргументы по умолчанию. Аргументы, указанные после имени образа в `docker run`, заменяют `CMD`, но не `ENTRYPOINT`. Для программы командной строки удобно задать `ENTRYPOINT ["quant-cli"]`: тогда контейнер ведёт себя как сама программа.

```
docker run --rm quant-cli:multi --symbol ALFA --count 500 --sma 6
# аргументы после имени образа попадают в quant-cli
```

4.5. Многоархитектурные образы

Облачные инстансы бывают двух архитектур: x86-64 и arm64 (процессоры собственной разработки поставщика). Образ, собранный на x86-64, на инстансе arm64 не запустится: несовпадение архитектуры видно сразу по ошибке `exec format error`.

Решение — **список манифестов**: одно имя образа, за которым скрываются несколько образов под разные архитектуры. Исполнитель контейнеров запрашивает образ и получает тот вариант, который соответствует его платформе. Собирается такой образ сборщиком с поддержкой нескольких платформ.

Сборка многоархитектурного образа

```
# сборка образа сразу под две архитектуры
docker buildx build --platform linux/amd64,linux/arm64 \
  -t quantflow/quant-cli:1.0.0 -f docker/Dockerfile.multi --push .

# что получилось: один тег, два образа
docker buildx imagetools inspect quantflow/quant-cli:1.0.0
```

Архитектура	Где встречается	На что влияет
x86-64 (amd64)	большинство инстансов, ноутбуки на Intel и AMD	готовые пакеты и образы
arm64 (aarch64)	инстансы на процессорах поставщика, ноутбуки Apple	цена вычислений, энергопотребление

Обратите внимание. Многоархитектурный образ — не роскошь: обучающиеся с ноутбуками на процессоре Apple собирают arm64-образы, а эмулятор и учебные стенды нередко работают на x86-64. Явное указание платформы при сборке избавляет от долгих разборов, почему «у меня работает, а на стенде — нет».

4.6. Возможности современного сборщика

Сборка образа давно не сводится к последовательному выполнению инструкций. Современный сборщик строит граф зависимостей этапов и выполняет независимые этапы **параллельно**, а также умеет то, чего нельзя было сделать простым кэшем слоёв.

Возможность	Что даёт	Пример применения
Параллельные этапы	независимые этапы собираются одновременно	сборка приложения и подготовка данных
Кэш-монтирование	каталог кэша сохраняется между сборками, но не попадает в образ	кэш пакетов и объектных файлов
Секреты сборки	значение доступно только во время инструкции и не остаётся в слое	токен доступа к частному репозиторию
Монтирование исходных текстов	файлы доступны этапу без копирования в слой	проверка и тесты

Кэш-монтирование ускоряет повторную сборку и не увеличивает образ

```
# syntax=docker/dockerfile:1
FROM ubuntu:24.04 AS build
RUN --mount=type=cache,target=/var/cache/apt \
    apt-get update && apt-get install -y --no-install-recommends build-essential cmake
WORKDIR /src
COPY . .
RUN --mount=type=cache,target=/src/build \
    cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j \
 && cp build/apps/quant-cli/quant-cli /usr/local/bin/
```

Важно. Кэш-монтирование не заменяет правильного порядка инструкций: кэш ускоряет повторную сборку на той же машине, а порядок инструкций определяет, какие слои переиспользуются везде, включая конвейер сборки и другие машины.

5. Многоэтапная сборка и минимальные образы



Рисунок 5.1 — Многоэтапная сборка

Для компилируемого языка образ, в котором велась сборка, содержит компилятор, заголовочные файлы и промежуточные объектные файлы — всё это не нужно для запуска. **Многоэтапная сборка** описывает несколько этапов `FROM`; в итоговый образ инструкцией `COPY --from` переносится только результат.

Ещё меньше образ становится при статической компоновке: программа не зависит от системных библиотек, и её можно поместить в образ **distroless**, где нет ни оболочки, ни пакетного менеджера. Ниже — размеры образов с одной и той же программой, измеренные на архитектуре arm64.

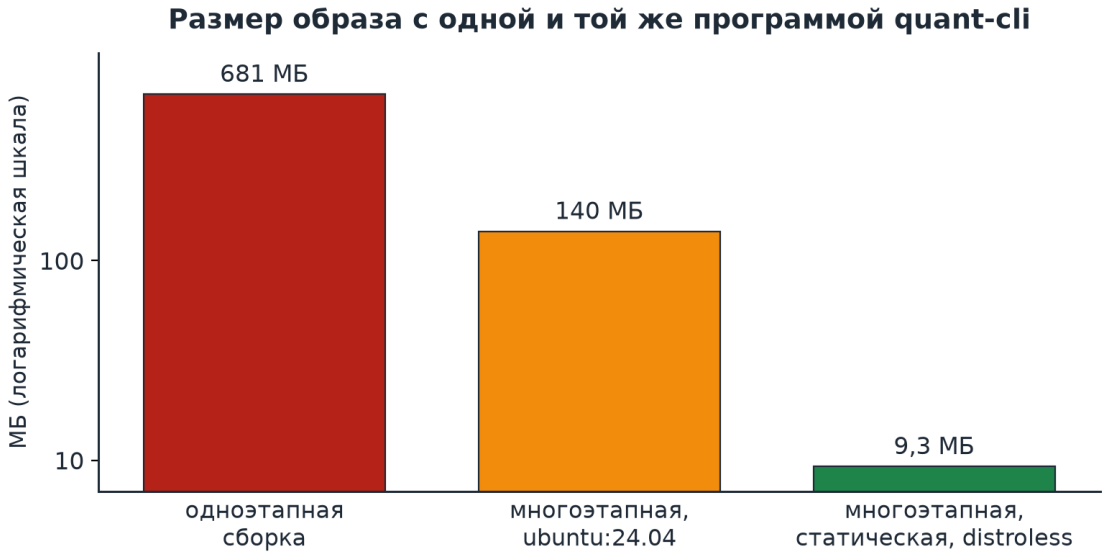


Рисунок 5.2 — Размер образа при трёх способах сборки

Способ	Размер	Что внутри	Диагностика
Одноэтапная сборка	681 МБ	компилятор, исходные тексты, объектные файлы, программа	доступна оболочка
Многоэтапная, ubuntu:24.04	140 МБ	система и программа	доступна оболочка

Способ	Размер	Что внутри	Диагностика
Многоэтапная, статическая, distroless	9 МБ	только программа	оболочки нет

Размер образа влияет на время развёртывания (образ загружается на каждый узел), на стоимость хранения в реестре и на поверхность атаки: чем меньше в образе программ, тем меньше в нём уязвимостей.

5.1. Происхождение образа и его проверка

Образ — это код, который будет выполнен на рабочем узле. Поэтому к нему применимы те же вопросы, что и к любому поставляемому коду: откуда он взят, что внутри, не подменён ли он по дороге.

— **Фиксация базового образа.** `ubuntu:24.04` вместо `ubuntu:latest`, а в строгом варианте — ссылка на дайджест базового образа: тогда пересборка даёт тот же результат;

— **Перечень содержимого** (список пакетов и их версий, SBOM) сохраняется вместе с образом и позволяет ответить, затронут ли образ новой уязвимостью, не пересобирая его;

— **Подпись образа** подтверждает, что образ собран вашим конвейером, а не подменён в реестре; проверка подписи выполняется перед запуском;

— **Воспроизводимость сборки**: одинаковые исходные тексты и одинаковый базовый образ должны давать одинаковый результат — иначе сравнивать сборки бессмысленно.

Эти меры образуют то, что называют защитой цепочки поставки программного обеспечения. В курсе они применяются постепенно: фиксация версий — с этого занятия, сканирование — на занятии по конвейеру, подпись и проверка — при обсуждении безопасности.

6. Запуск контейнеров

6.1. Основные параметры

Параметр	Назначение
`-d`	запуск в фоне; идентификатор контейнера выводится на экран
`--rm`	удалить контейнер после завершения
`--name имя`	имя контейнера (иначе назначается случайное)
`-p 8080:8080`	опубликовать порт контейнера на хосте
`-e KEY=value`	переменная окружения
`-v источник:/путь`	том или каталог хоста
`--cpus`, `--memory`	пределы ресурсов
`--user uid:gid`	запуск от имени заданного пользователя
`--network имя`	подключение к сети Docker (занятие 6)

6.2. Тома и монтирование

Контейнер по своей природе недолговечен: его слой записи удаляется вместе с контейнером. Данные, которые должны пережить контейнер, размещают вне его.

— **монтирование каталога хоста** (`-v /путь/на/хосте:/data`) — файлы видны и хосту, и контейнеру; права берутся из файловой системы хоста;

— **именованный том** (`-v имя:/data`) — областью управляет Docker; при первом подключении том наследует содержимое и права каталога из образа.

Важно. Если процесс в контейнере работает от непривилегированного пользователя, а каталог принадлежит другому владельцу, запись завершится ошибкой Permission denied. Решения: запуск с `--user $(id -u):$(id -g)`, изменение прав каталога на хосте или именованный том, каталог которого создан в образе с нужным владельцем: `RUN useradd --system --uid 10001 quant && mkdir /data && chown quant /data`.

6.3. Ограничение ресурсов

Без ограничений контейнер может занять всю память узла. Пределы задаются параметрами `--cpus` и `--memory`; при превышении предела памяти ядро прекращает работу процесса (состояние OOMKilled). В оркестраторах те же пределы задаются в описании задачи или пода — это будет на занятиях 10 и 17.

6.4. Диагностика

```
docker ps -a                        # список контейнеров и их состояние
docker logs -f <имя>                # журнал (стандартный вывод процесса)
docker exec -it <имя> sh            # оболочка внутри работающего контейнера
docker inspect <имя> | head -40     # полные метаданные
docker history <образ>              # слои образа и их размеры
docker stats --no-stream            # потребление ресурсов
```

6.5. Процесс номер один: сигналы и корректное завершение

Внутри контейнера главный процесс получает номер 1. Это не формальность: ядро относится к процессу с номером 1 особым образом — необработанные сигналы по умолчанию **не завершают** его, а завершившиеся дочерние процессы должны им подбираться.

Отсюда три практических следствия, которые проявляются именно в контейнерах:

1. если главный процесс не обрабатывает сигнал завершения, остановка контейнера превращается в принудительное убийство по истечении времени ожидания (по умолчанию 10 секунд);
2. если главным процессом сделан сценарий оболочки, сигнал получает оболочка, а не приложение: приложение убивается без возможности завершить работу;
3. если приложение порождает дочерние процессы и не подбирает их, в контейнере накапливаются «зомби»-процессы.

Форма записи инструкции определяет, кто получит сигнал завершения

```
# плохо: сигнал получает /bin/sh, приложение о нём не узнает
CMD quant-cli --symbol ALFA --count 1000

# хорошо: приложение — первый процесс и получает сигналы напрямую
ENTRYPOINT ["quant-cli"]
CMD ["--symbol", "ALFA", "--count", "1000"]

# если нужен сценарий-обёртка — передать управление приложению
exec quant-cli "$@"
```

Корректное завершение важно не только для аккуратности. При обновлении службы оркестратор останавливает старые задачи: приложение, умеющее завершаться, успевает дообслужить текущие запросы и закрыть соединения, а не умеющее — обрывает их, и пользователь видит ошибку. То же относится к прерываемым инстансам, где на завершение отводится две минуты.

Что происходит	Сигнал	Как должно вести себя приложение
Остановка контейнера	завершение (SIGTERM)	перестать принимать новое, дообслужить текущее, выйти
Истекло время ожидания	убийство (SIGKILL)	обработать невозможно: работа прерывается
Обновление службы	завершение старых задач	закрыть соединения, снять регистрацию
Изъятие прерываемого ресурса	уведомление и завершение	сохранить контрольную точку и выйти

6.6. Ограничение процессора: доли, квоты и торможение

Ограничение памяти работает жёстко: превышение предела приводит к остановке процесса ядром (состояние OOMKilled). Ограничение процессора устроено иначе — оно **замедляет** программу, а не останавливает её, и понимать разницу необходимо.

— **доля** (`--cpu-shares`) — относительный вес при конкуренции: если конкуренции нет, контейнер получает всё доступное время;

— **квота** (`--cpus`) — жёсткий предел: за каждый период планировщика контейнер получает не больше заданной доли процессорного времени, даже если процессор свободен.

Когда квота исчерпана, процесс **тормозится**: он остаётся готовым к выполнению, но не получает процессор до начала следующего периода. Внешне это выглядит как необъяснимое увеличение времени ответа при низкой средней загрузке — классическая загадка эксплуатации.

Пределы ресурсов и их наблюдение

```
docker run --rm --cpus 0.5 --memory 128m quantflow/quant-cli:1.0 --count 200000
docker stats --no-stream            # доля процессора и память контейнера
docker inspect --format '{{.State.OOMKilled}} {{.State.ExitCode}}' имя_контейнера
```

Показатель	О чём говорит	Что делать
Доля процессора у предела	квоты не хватает	увеличить квоту или распараллелить работу

Показатель	О чём говорит	Что делать
Счётчик торможений растёт	процесс упирается в квоту периодически	поднять квоту, проверить всплески
Память у предела	скоро остановка ядром	увеличить предел или найти утечку
Состояние OOMKilled	процесс остановлен ядром	увеличить предел памяти, проверить потребление
Код выхода 137	процесс убит принудительно	проверить обработку сигнала завершения

Обратите внимание. В облаке те же два параметра задаются на уровне задачи, а их значения выбираются по измерениям: сначала измеряется потребление под нагрузкой, потом назначается предел с запасом. Назначение предела «на глаз» приводит либо к переплате, либо к торможению под нагрузкой.

7. Amazon ECR: реестр образов

Собранный образ нужно где-то хранить и откуда-то получать при запуске. Эту задачу решает **реестр** — служба хранения образов, которая раздаёт слои по запросу и ведёт учёт версий.

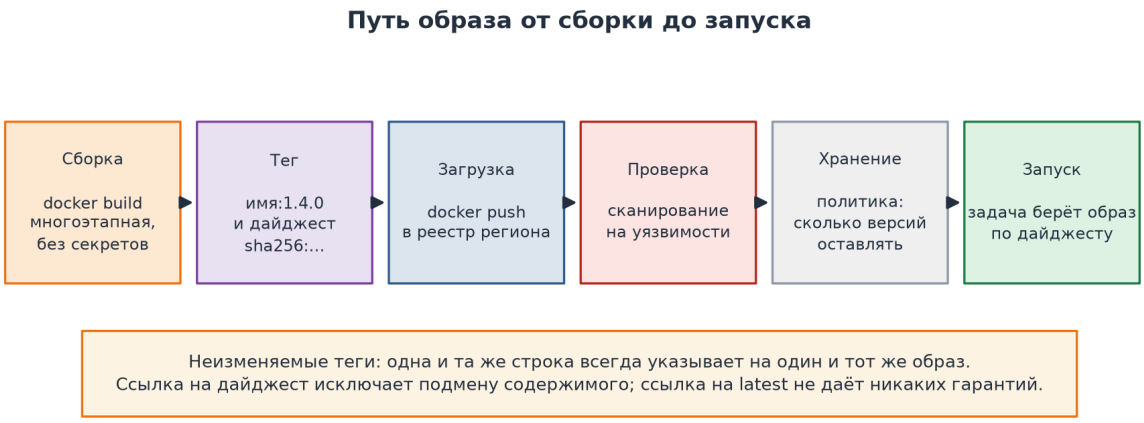


Рисунок 7.1 — Путь образа от сборки до запуска

7.1. Имя образа и дайджест

Полное имя образа имеет вид `реестр/пространство/имя:тег`. Если реестр не указан, используется реестр по умолчанию. Тег — **изменяемая** ссылка: одна и та же строка сегодня и завтра может указывать на разные образы. Дайджест `sha256:…` вычисляется по содержимому манифеста и однозначно определяет образ.

Ссылки на образ и получение дайджеста

```
# три способа сослаться на один и тот же образ
quantflow/indicators-api:latest            # ссылка меняется при каждой публикации
quantflow/indicators-api:1.4.0             # неизменяемый тег версии
quantflow/indicators-api@sha256:9f2c…4e1b  # ссылка на содержимое, подмена невозможна

docker inspect --format '{{index .RepoDigests 0}}' quantflow/indicators-api:1.4.0
```

Способ ссылки	Воспроизводимость	Где применяется
`latest`	отсутствует	локальные эксперименты
тег версии	есть, если тег неизменяемый	выкладка приложения, откат на версию
дайджест	полная	промышленная эксплуатация, описание инфраструктуры

Важно. Развёртывание по тегу `latest` делает систему невоспроизводимой: два узла, запущенные с разницей в минуту, могут работать на разных версиях приложения, а откат становится невозможным — прежнего образа под тем же тегом уже нет.

7.2. Реестр рядом с нагрузкой

Реестр облачного поставщика располагается в том же регионе, что и вычислительные ресурсы. Это важно по трём причинам: загрузка слоёв идёт по внутренней сети (быстрее и без платы за исходящий трафик), права выдаются той же системой управления доступом, что и для остальных ресурсов, а из частной подсети образ доступен без выхода в интернет — через конечную точку сервиса внутри сети.

Реестр	Права доступа	Особенности
Публичный реестр сообщества	учётная запись реестра	ограничение частоты загрузок, внешний трафик
Реестр системы контроля версий	токен репозитория	удобен для конвейера сборки
Реестр облачного поставщика	роли и политики облака	внутренняя сеть, сканирование, политики хранения

7.3. Сканирование образов

Реестр может проверять каждый загруженный образ на известные уязвимости: содержимое слоёв сопоставляется с базами уязвимостей по версиям пакетов. Результат — перечень находок с уровнями серьёзности и указанием пакета и исправляющей версии.

— сканирование **при загрузке** даёт результат один раз, на момент публикации;

— **постоянное** сканирование перепроверяет уже лежащие образы, когда в базу добавляются новые уязвимости — образ, безопасный в марте, может стать уязвимым в апреле без единого изменения;

— находки уровня «критический» и «высокий» обычно блокируют выкладку в конвейере (занятие 13);

— часть находок относится к пакетам, которые приложение не использует, — их отмечают как принятый риск с обоснованием, а не игнорируют молча.

Отсюда практическое правило: **регулярная пересборка образа — это мера безопасности**. Приложение не менялось, но базовый образ обновился, и уязвимость исчезла.

7.4. Политика хранения

Каждая сборка добавляет в реестр новый образ. За полгода в репозитории накапливаются сотни версий, из которых нужны единицы. **Политика хранения** описывает, что удалять: например, «оставлять 10 последних образов с тегом версии и удалять образы без тегов старше 7 дней».

Правило формулируется так, чтобы нельзя было удалить работающую версию. Обычно это достигается двумя условиями: хранить достаточное число последних версий и никогда не удалять образы, на которые ссылаются работающие службы.

8. Amazon ECS: оркестрация контейнеров в облаке

Запуск контейнера командой `docker run` на одной машине не отвечает на вопросы: что делать при отказе машины, как обновить приложение без перерыва, как распределить нагрузку между копиями. На эти вопросы отвечает **оркестратор**.

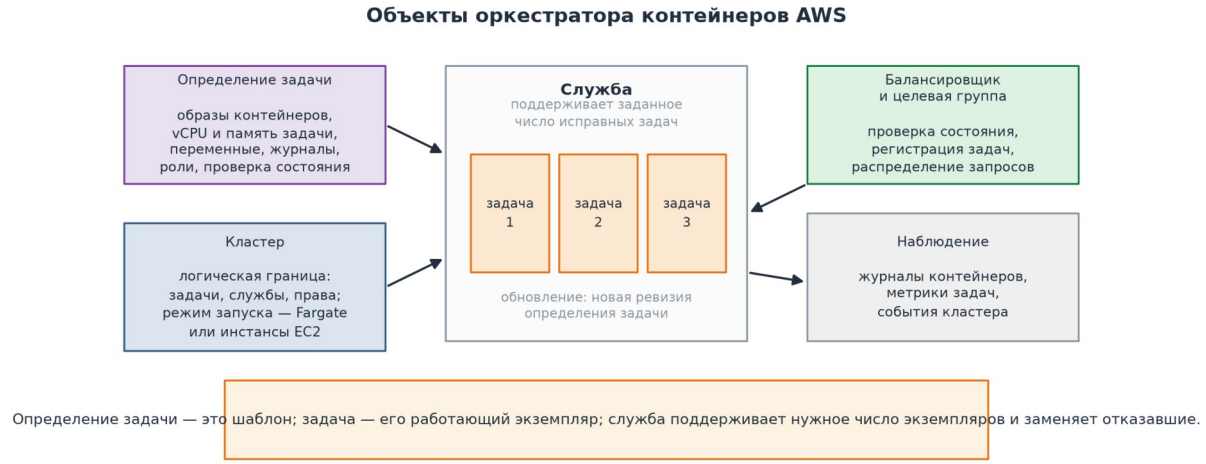


Рисунок 8.1 — Объекты оркестратора контейнеров

8.1. Определение задачи, задача и служба

Объект	Что описывает	Аналог из занятия
Определение задачи	образы, ресурсы, переменные, журналы, роли, проверка состояния	Dockerfile описывает образ, определение задачи — запуск
Задача	работающий экземпляр определения задачи	контейнер, запущенный `docker run`
Служба	поддерживает заданное число исправных задач	перезапуск контейнера вручную
Кластер	логическая граница: задачи, службы, права	хост, на котором запущены контейнеры

Определение задачи версионируется: каждое изменение создаёт новую **ревизию**. Служба всегда ссылается на конкретную ревизию, поэтому обновление приложения — это переключение службы на новую ревизию, а откат — возврат на предыдущую. Это тот же принцип неизменяемости, что и у образов: меняется не содержимое, а ссылка.

8.2. Запуск без управления серверами

В режиме запуска без управления серверами каждая задача выполняется в отдельной облегчённой виртуальной машине, которую создаёт и обслуживает поставщик. Потребитель не видит узлов и не обновляет их; он задаёт процессор и память **на уровне задачи**. Допустимые сочетания фиксированы.

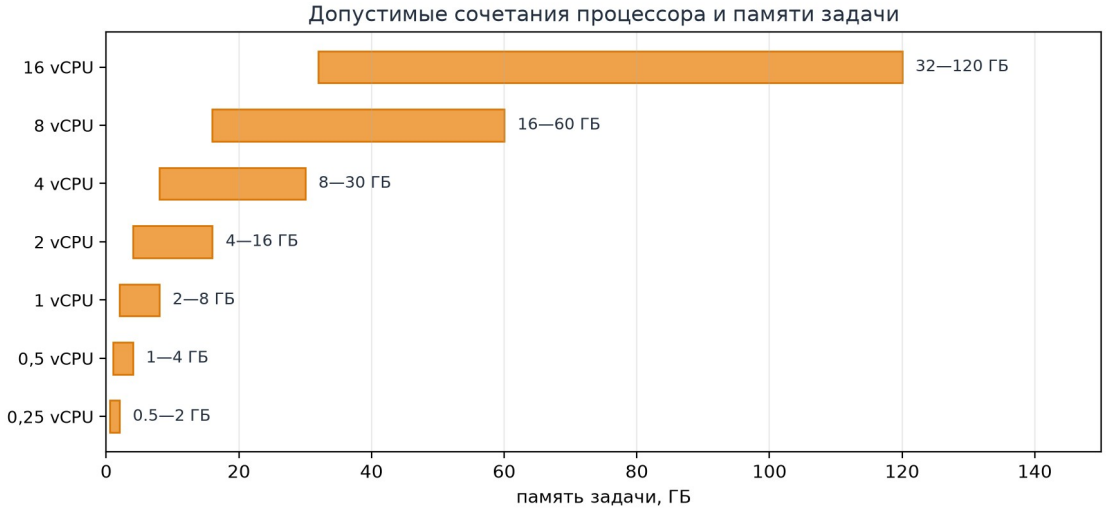


Рисунок 8.2 — Допустимые сочетания процессора и памяти задачи

Процессор задачи	Допустимая память	Шаг
0,25 vCPU	512 МиБ, 1 ГБ, 2 ГБ	фиксированные значения
0,5 vCPU	от 1 до 4 ГБ	1 ГБ
1 vCPU	от 2 до 8 ГБ	1 ГБ
2 vCPU	от 4 до 16 ГБ	1 ГБ
4 vCPU	от 8 до 30 ГБ	1 ГБ
8 vCPU	от 16 до 60 ГБ	4 ГБ
16 vCPU	от 32 до 120 ГБ	8 ГБ

Сочетания объясняют типичную ошибку планирования: нельзя заказать «0,25 vCPU и 8 ГБ памяти». Если приложению нужна память, придётся взять и процессор, и это отразится на счёте. Поэтому потребление памяти измеряют до выбора размера задачи, а не после.

Сеть задачи устроена так же, как у виртуальной машины: задача получает **собственный сетевой интерфейс** в подсети и собственные правила группы безопасности. Отсюда следствие: задачи обращаются друг к другу по адресам сети, а не через общий сетевой стек узла, и каждой задаче нужен маршрут к реестру образов — либо через шлюз, либо через конечную точку сервиса.

Обратите внимание. Запуск задачи включает загрузку образа. Крупные образы заметно увеличивают время запуска, поэтому ленивая загрузка (образ начинает работать, не дожидаясь полной загрузки слоёв) применяется к образам от четверти гигабайта. Это ещё один аргумент за многоэтапную сборку: маленький образ запускается быстрее.

8.3. Узлы или без узлов: как выбирать

Признак	Задачи на своих инстансах	Запуск без управления серверами
Кто обслуживает узлы	потребитель: обновления, ёмкость	поставщик
Оплата	за инстансы независимо от загрузки	за процессор и память задачи, посекундно
Плотность размещения	несколько задач на инстанс	одна задача — одна изолированная среда
Специальные требования	драйверы, ускорители, особые ядра	ограничены поддержкой сервиса
Когда выгоднее	постоянная высокая загрузка, планы экономии	переменная нагрузка, мало задач

Разобранный пример. Служба работает круглосуточно и занимает 0,5 vCPU и 1 ГБ памяти, реплик три. При запуске без управления серверами оплачивается 1,5 vCPU и 3 ГБ непрерывно. Тот же объём умещается на одном небольшом инстансе, который при постоянной нагрузке выкупается планом экономии. Наоборот, если та же служба нужна три часа в сутки, узлы простаивают 87 % времени, и запуск без управления серверами дешевле.

8.4. Другие способы запустить образ

Способ	Что даёт	Ограничения
Amazon EKS (управляемый Kubernetes)	переносимость описаний, богатая экосистема	выше сложность эксплуатации
Функция из образа	запуск по событию, оплата за время работы	образ до 10 ГБ, работа до 900 с
Сервис веб-приложений из образа	минимум настроек, маршрут и масштабирование сразу	меньше контроля
Пакетный сервис	очередь заданий, прерываемые ресурсы	не для служб, отвечающих на запросы

Линия ML. Сервис применения модели упаковывается в контейнер по тем же правилам и запускается либо как служба (постоянная нагрузка, низкая задержка), либо как пакетное задание (периодический расчёт по расписанию). Выбор делается по профилю обращений, а не по типу модели. Задания с пометкой «Линия ML» рассчитаны на обучающихся, которые параллельно изучают дисциплину «Машинное обучение в облаке»: результат такого задания засчитывается в обеих дисциплинах и не требует двойной работы. Остальные обучающиеся выполняют то же задание с готовым эталонным компонентом, описанным в тексте; знание машинного обучения для этого не требуется.

9. Эксплуатация контейнеров в облаке

9.1. Две роли: кто запускает и кто работает

У задачи в облаке две разные роли, и их путают чаще всего. **Роль исполнения** нужна инфраструктуре: получить образ из реестра, записать журналы, прочитать секреты при запуске. **Роль задачи** нужна приложению: обратиться к хранилищу, очереди, таблице. Первая действует до старта контейнера, вторая — внутри него.

Роль	Кому выдаётся	Типичные права
Роль исполнения	инфраструктуре запуска	получение образа, запись журналов, чтение секрета
Роль задачи	процессу внутри контейнера	чтение и запись в хранилище, отправка в очередь

Разделение даёт практический эффект: приложение не получает права на реестр образов, а инфраструктура — прав на данные приложения. Нарушение любого из процессов ограничено своей областью.

9.2. Журналы

Контейнер пишет в стандартный вывод, а драйвер журналирования перенаправляет поток в службу журналов: там записи группируются по службе и потоку, хранятся заданный срок и доступны поиску. Правила остаются теми же, что и для локального контейнера: писать в стандартный вывод, не заводить файлов журнала внутри контейнера, выдавать по одной записи на событие в структурированном виде.

Структурированная запись журнала: её можно фильтровать и агрегировать

```
{"ts":"2026-09-23T10:15:02Z","level":"info","service":"indicators-api",
 "instance":"a1b2c3","route":"/v1/indicators","status":200,"ms":12.4}
```

9.3. Секреты

Пароли, ключи и токены не попадают ни в образ, ни в определение задачи в открытом виде. В определении задачи указывается **ссылка** на секрет в хранилище секретов, а значение подставляется в переменную окружения при запуске. Так секрет не виден в описании инфраструктуры, не сохраняется в слоях образа и может быть заменён без пересборки.

Важно. Значения, переданные инструкцией ENV при сборке, видны в метаданных образа любому, кто может получить образ. Удаление файла в следующем слое не удаляет его из предыдущего: слой остаётся в образе целиком. Секрет, однажды попавший в образ, считается скомпрометированным.

9.4. Проверка состояния и обновление службы

Проверка состояния объявляется в определении задачи и выполняется инфраструктурой: неисправная задача исключается из обслуживания и заменяется. При обновлении служба запускает задачи новой ревизии, дожидается их готовности и только потом останавливает старые. Если новая ревизия не проходит проверку, обновление останавливается — и это правильное поведение: перерыва в обслуживании нет, а неисправная версия запросов не получает.

Отсюда требование к приложению: проверка состояния должна отражать способность обслуживать запросы, а не факт запуска процесса. Проверка, которая всегда возвращает «готов», превращает механизм самовосстановления в фикцию.

9.5. Хранилище для задач

Контейнер по умолчанию пишет во временный слой, который исчезает вместе с задачей. Когда данные должны пережить перезапуск, к задаче подключают хранилище — и выбор здесь тот же, что для виртуальной машины, только на уровне задачи.

Вид хранилища	Что даёт	Когда применяется
Временное хранилище задачи	место для файлов на время работы	распаковка, кэш, промежуточные файлы
Сетевая файловая система	общий доступ нескольких задач, рост по мере записи	общие файлы, модели, выгрузки
Блочный том	низкая задержка, высокая производительность	база данных, журнал операций одной задачи
Объектное хранилище	неограниченный объём, доступ по ключу	результаты, наборы данных, архивы

Правило выбора совпадает с правилом для локальной разработки: **состояние не хранится внутри контейнера**. Если задачу можно остановить и запустить заново без потери данных, приложение готово к оркестратору; если нет — оно к нему не готово, сколько бы реплик ни описали.

9.6. Что измерять у контейнера

Контейнер закрывает приложение от посторонних глаз, поэтому набор измеряемых величин фиксируется заранее — иначе разбор сбоя превращается в догадки.

Величина	Откуда берётся	О чём говорит
Доля процессора	счётчики контрольной группы	приближение к квоте, необходимость масштабирования
Число торможений	счётчики контрольной группы	нехватка квоты при всплесках
Память	счётчики контрольной группы	приближение к пределу, утечка
Перезапуски задачи	оркестратор	падения приложения, неудачные проверки состояния
Время ответа и доля ошибок	приложение	качество обслуживания с точки зрения клиента
Время запуска	оркестратор	влияние размера образа и прогрева на масштабирование

Первые три величины собираются инфраструктурой без участия приложения, последние — только если приложение их сообщает. Поэтому вывод структурированной записи журнала на каждый запрос и простейшая страница метрик — это часть приложения, а не дополнительная опция.

10. Эталонная архитектура контейнерного приложения

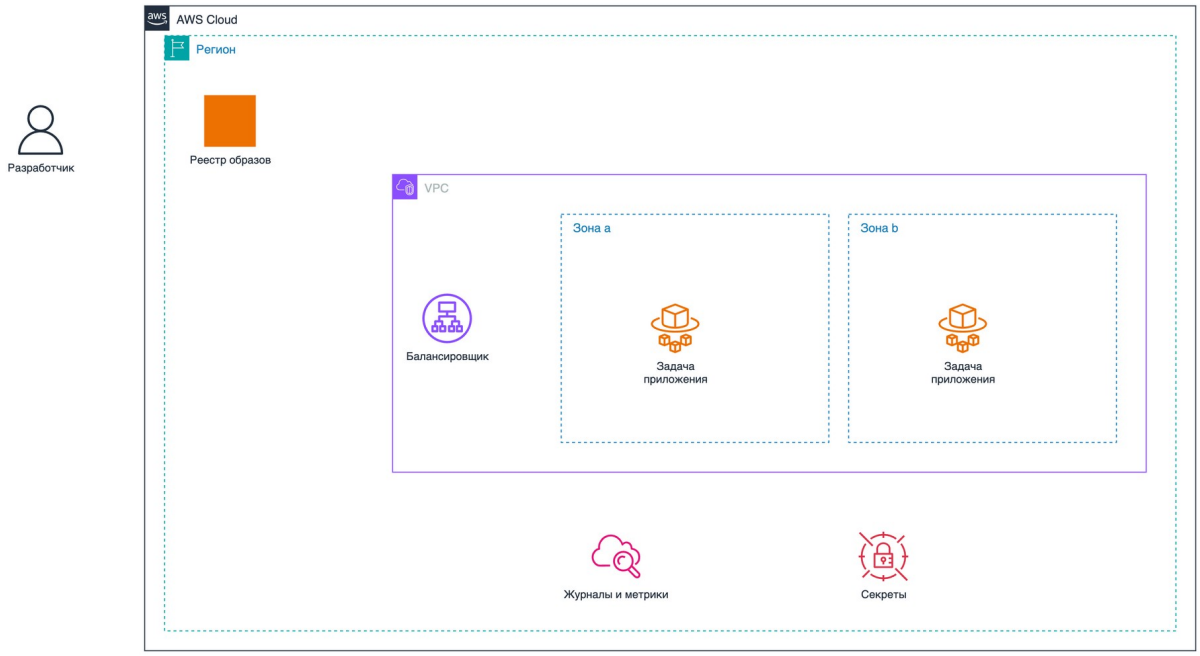


Рисунок 10.1 — Контейнерное приложение в облаке

1. разработчик собирает образ и публикует его в реестре региона с неизменяемым тегом;
2. реестр проверяет образ на уязвимости и хранит ограниченное число версий;
3. служба запускает задачи в двух зонах доступности по указанной ревизии определения задачи;
4. балансировщик распределяет запросы между исправными задачами;
5. журналы и метрики уходят в службу наблюдения, секреты подставляются при запуске.

Эта схема — прямое продолжение локальной работы: то, что в лабораторной работе делается командами `docker build` и `docker run`, в облаке описывается определением задачи и службой. Именно поэтому образ, собранный на занятии, запускается в облаке без изменений.

Что сделано локально	Что соответствует в облаке
`docker build -t имя:тег`	сборка образа в конвейере и публикация в реестр
`docker run -p 8080:8080`	определение задачи с отображением порта и сетевым интерфейсом
`--memory`, `--cpus`	процессор и память на уровне задачи
`HEALTHCHECK`	проверка состояния задачи и целевой группы балансировщика
`-v том:/data`	подключение сетевого хранилища к задаче
`docker logs`	журналы задачи в службе журналов
перезапуск контейнера вручную	служба поддерживает заданное число исправных задач

11. Способы запуска контейнера в облаке

Собранный образ можно запустить в облаке несколькими способами, и они отличаются не возможностями, а тем, **что именно берёт на себя поставщик** и **за что выставляется счёт**. Выбор делается до написания описания развёртывания, потому что переход между способами требует переделки конфигурации.

Способ запуска	Чем управляет обучающийся	Единица оплаты	Когда уместен
Amazon ECS на Amazon EC2 (оркестратор задач на своих узлах)	тип и число узлов, размещение задач	инстансы, пока они запущены	постоянная нагрузка, нужен контроль над узлом
Amazon ECS с AWS Fargate (оркестратор задач без узлов)	только описание задачи: процессор, память, образ	vCPU·час и ГиБ·час задачи	переменная нагрузка, малое число задач
Amazon EKS (управляемый Kubernetes)	манифесты, узлы или профили без узлов	плата за управление кластером и за узлы	нужен Kubernetes: операторы, сложные политики размещения
AWS App Runner (запуск веб-приложения из образа)	образ и параметры масштабирования	вычислительные ресурсы экземпляров	одиночная веб-служба без собственной сети
AWS Lambda с образом контейнера	код и параметры вызова	вызовы и время работы	редкие обращения, обработка событий
AWS Batch (пакетные задания)	очередь заданий и вычислительная среда	инстансы под задания	счётные задачи, обучение, расчёты по расписанию

Первые два способа используют один и тот же формат **определения задачи**: описание контейнеров, их образов, портов, переменных окружения, ограничений процессора и памяти, проверки состояния и роли доступа. Переход «свои узлы → без узлов» сводится к смене одного параметра, поэтому оба способа рассматриваются как один с двумя режимами.

11.1. Ограничения, которые определяют выбор

Ограничение	Значение	Следствие для выбора
Предельное время одного вызова функции	900 с (15 минут)	длительная обработка потока — не для функции
Память функции	от 128 МБ до 10 240 МБ	вычисления с большой памятью — в задачу оркестратора
Размер образа функции	до 10 ГБ	образ с библиотеками машинного обучения помещается, но запускается медленнее
Соответствие памяти и процессора функции	1769 МБ ≈ 1 vCPU	процессор не задаётся отдельно: он растёт вместе с памятью
Сочетания процессора и памяти задачи без узлов	фиксированный набор	произвольные значения задать нельзя, см. таблицу ниже

Допустимые сочетания процессора и памяти задачи, запускаемой без управления серверами

Процессор задачи	Допустимая память
0,25 vCPU	512 МиБ, 1 ГБ, 2 ГБ
0,5 vCPU	от 1 до 4 ГБ, шаг 1 ГБ
1 vCPU	от 2 до 8 ГБ, шаг 1 ГБ
2 vCPU	от 4 до 16 ГБ, шаг 1 ГБ
4 vCPU	от 8 до 30 ГБ, шаг 1 ГБ
8 vCPU	от 16 до 60 ГБ, шаг 4 ГБ
16 vCPU	от 32 до 120 ГБ, шаг 8 ГБ

Из таблицы следует практическое правило: измеренное потребление округляется **вверх** до ближайшего допустимого сочетания. Задача, которой нужно 1,2 ГиБ памяти при 0,4 vCPU, получит 0,5 vCPU и 2 ГБ — и оплачиваться будет именно это, а не измеренное значение.

11.2. Разобранный пример выбора

Условие. Сервис расчёта индикаторов обрабатывает 30 запросов в секунду, каждый занимает 25 мс процессорного времени и 180 МиБ памяти. Пик приходится на рабочие часы, ночью нагрузки почти нет. Что выбрать?

1. Средняя занятость процессора: 30 × 0,025 = 0,75 одновременно работающих обработчика.
2. Требование к памяти на копию: 180 МиБ + запас 40 % ≈ 252 МиБ.
3. Функция: 30 вызовов в секунду — это около 78 млн вызовов в месяц, каждый со своим временем работы; при постоянной дневной нагрузке это дороже задачи и даёт холодные старты на пиках.
4. Задача без узлов: ближайшее сочетание — 0,5 vCPU и 1 ГБ; двух копий достаточно и при отказе одной из них.
5. Ночное снижение: число копий уменьшается автоматическим масштабированием до одной, останавливать полностью нельзя — нужен ответ на редкие запросы.
6. Ответ: две задачи 0,5 vCPU / 1 ГБ с масштабированием по загрузке процессора; функция остаётся вариантом для ночной обработки отчётов.

Замечание. Сравнение способов запуска всегда проводится при **одинаковом качестве обслуживания**. Функция без предварительно прогретых экземпляров отвечает медленнее на первом запросе, и если требование к времени ответа этого не допускает, сравнение по цене уже некорректно.

12. Стоимость контейнерной нагрузки

Стоимость запуска контейнера складывается из нескольких составляющих, и вычислительные ресурсы в ней не всегда главные. Ниже перечислено то, что тарифицируется вокруг одной работающей задачи.

Составляющая	Что оплачивается	На что влияет обучающийся
Вычисления	vCPU·час и ГиБ·час каждой задачи	размер задачи, число копий, время работы
Хранение образов	объём хранимых образов в реестре	размер образа, политика удаления старых
Передача образа	загрузка образа из реестра на узел	размер образа, кэш слоёв на узле
Балансировщик	час работы и объём обработанных запросов	число балансировщиков, схема
Журналы и метрики	объём принятых записей и срок хранения	уровень журналирования, срок
Трафик	исходящий трафик и обмен между зонами	размещение служб, схема обращений

12.1. Расчёт для опорной композиции

Пусть три копии сервиса работают непрерывно с размером задачи 0,5 vCPU и 1 ГБ. Учебные тарифы: 0,04048 USD за vCPU·час и 0,004445 USD за ГиБ·час.

1. Часы в месяце: 730.
2. Процессор: $3 \times 0{,}5 \times 730 \times 0{,}04048 \approx 44{,}33$ USD.
3. Память: $3 \times 1 \times 730 \times 0{,}004445 \approx 9{,}73$ USD.
4. Итого вычисления: ≈ 54,06 USD в месяц.
5. Балансировщик и журналы добавляют порядка 20—25 USD, то есть около трети счёта.

Расчёт показывает, что **память дешевле процессора примерно в девять раз** в пересчёте на единицу. Отсюда правило: сначала уменьшают заказанный процессор до измеренного значения, и только потом занимаются памятью.

12.2. Способы снижения стоимости

Способ	В чём состоит	Когда применим
Размер по измерению	заказывается измеренное потребление с запасом, а не «с большим запасом»	всегда; даёт самый большой эффект
Прерываемые задачи	задача может быть остановлена поставщиком при нехватке ёмкости	обработка очередей, пакетные расчёты, проверочные среды
План экономии вычислений	обязательство на постоянный уровень расходов, скидка до 66 %	постоянная нагрузка, известная на год вперёд
Остановка проверочных сред	среда создаётся и удаляется, а не работает постоянно	учебные и проверочные окружения
Уменьшение образа	меньше хранение, быстрее загрузка на узел	всегда
Уровень журналирования	отладочные записи отключаются в промышленной среде	когда объём журналов сравним с вычислениями

План экономии вычислений распространяется и на задачи без узлов, и на функции, и на инстансы: обязательство берётся в долларах в час, а не в конкретных ресурсах. Точка окупаемости считается так же, как на занятии 4: при скидке d обязательство выгодно, если ресурс используется более $(1 - d)$ времени.

Тарифы приведены для региона US East (N. Virginia) по состоянию на сентябрь 2026 года и используются для учебных расчётов; перед применением в работе они сверяются с калькулятором стоимости.

13. Разобранные примеры

Примеры разобраны полностью: условие, ход рассуждения, вычисления и вывод. Числа в них не «красивые» — они такие же, как в заданиях, чтобы порядок величин запоминался вместе со способом расчёта.

13.1. Почему образ такой большой

Условие. Одноэтапная сборка дала образ 681 МБ, многоэтапная — 140 МБ, а на минимальном базовом образе — 9,3 МБ. Из чего складывается разница?

1. В одноэтапном образе остаются компилятор, заголовочные файлы, системы сборки и кэш пакетов — около 540 МБ, которые нужны только во время сборки.
2. Многоэтапная сборка копирует в итоговый образ только исполняемый файл и оставляет базовый образ операционной системы — отсюда 140 МБ.
3. Минимальный базовый образ содержит лишь библиотеки времени выполнения, без оболочки и пакетного менеджера: 9,3 МБ.
4. Цена последнего варианта — невозможность зайти в контейнер оболочкой: диагностика выполняется снаружи, по журналам и метрикам.

Вывод. Размер образа определяется не программой, а тем, что осталось рядом с ней. Многоэтапная сборка убирает инструменты, минимальный базовый образ — всё остальное.

13.2. Порядок инструкций и кэш

Условие. В Dockerfile копирование исходных текстов стоит перед установкой зависимостей. Почему сборка каждый раз идёт долго и как это исправить?

1. Кэш слоёв действует до первой изменившейся инструкции: дальше всё пересобирается.
2. Копирование исходных текстов меняет слой при любой правке кода, поэтому установка зависимостей, стоящая после него, тоже выполняется заново.
3. Правильный порядок: сначала установка зависимостей (она меняется редко), затем копирование исходных текстов (оно меняется часто).
4. Эффект измеряется: в опорном проекте перестановка сокращает повторную сборку с минут до десятков секунд.

Вывод. Слои упорядочивают по частоте изменения: редко меняющееся — вниз, часто меняющееся — вверх.

13.3. Контейнер остановлен ядром

Условие. Контейнер запущен с пределом памяти 128 МБ и завершился с кодом 137. Что произошло и что делать?

1. Код 137 означает завершение сигналом 9 (128 + 9): процесс убит принудительно.
2. Признак `OOMKilled` в состоянии контейнера подтверждает, что причина — превышение предела памяти.
3. Ограничение памяти — жёсткое: процесс не замедляется, а останавливается.
4. Правильные действия: измерить фактическое потребление под нагрузкой, назначить предел с запасом 30—50 %, а при неожиданно большом потреблении — искать утечку, а не поднимать предел.

Вывод. Предел памяти назначают по измерению, а не по догадке; код 137 — самый частый и самый объяснимый отказ контейнера.

13.4. Права на файлы в томе

Условие. Контейнер работает от непривилегированного пользователя и не может писать в подключённый каталог хоста. Почему и как решить?

1. Владелец каталога на хосте — пользователь с одним числовым идентификатором, а процесс в контейнере работает под другим.
2. Ядро сравнивает числовые идентификаторы, а не имена: имя пользователя внутри контейнера хостовой системе неизвестно.
3. Решения: задать владельца каталога, совпадающего с идентификатором процесса; запускать контейнер с нужным идентификатором; использовать именованный том вместо каталога хоста.
4. Запуск от суперпользователя «чтобы заработало» решением не является: он снимает изоляцию, ради которой контейнер и нужен.

Вывод. Права на томах — вопрос числовых идентификаторов. Именованные тома избавляют от него, поэтому их предпочитают для данных.

13.5. Сколько времени занимает загрузка образа на узлы

Условие. Образ службы состоит из трёх слоёв: базовый — 120 МиБ, слой зависимостей — 260 МиБ, слой приложения — 8 МиБ. Скорость загрузки из реестра — 40 МиБ/с. Служба разворачивается на четырёх узлах, ни на одном из которых слоёв ещё нет. Сколько времени займёт загрузка и что изменится при выпуске новой версии приложения?

1. Размер образа равен сумме слоёв: 120 + 260 + 8 = 388 МиБ.
2. Загрузка на узел «с нуля»: 388 ÷ 40 = 9,7 с. На четыре узла — 4 × 9,7 = 38,8 с, если узлы загружают слои последовательно из одного реестра.
3. При выпуске новой версии меняется только слой приложения: 8 ÷ 40 = 0,2 с на узел, 0,8 с на все четыре. Базовый слой и слой зависимостей берутся из локального кэша узла.
4. Экономия от неизменности двух первых слоёв: (9,7 − 0,2) ÷ 9,7 = 97,9 % времени загрузки.

Вывод. Время выкладки новой версии определяется не размером образа, а размером изменившегося слоя. Поэтому редко меняющиеся зависимости размещают в нижних слоях, а исходные тексты приложения — в верхних: тот же приём, что и для кэша сборки, но выгода проявляется уже при развёртывании.

14. Задачи для самопроверки

Задачи решаются без компьютера, по формулам и правилам занятия. Ответы приведены сразу после условий: сначала решите, потом сверьтесь.

14.1. Условия

1. Какой слой изменится, если поправить одну строку исходного текста при правильном порядке инструкций?
2. Образ 681 МБ и 140 МБ содержат одну и ту же программу. Что именно занимает разницу?
3. Контейнер завершился с кодом 137. Что проверить в первую очередь?
4. Чем ограничение процессора отличается от ограничения памяти по последствиям?
5. Почему инструкция с секретом в переменной окружения опасна даже после удаления файла?
6. Что произойдёт при остановке контейнера, если данные лежат в его файловой системе?
7. Зачем в образе непривилегированный пользователь, если контейнер и так изолирован?
8. Что даёт инструкция проверки состояния в образе?
9. Почему тег latest не годится для описания развёртывания?
10. Образ 400 МБ загружается по сети со скоростью 50 МБ/с. Сколько займёт запуск на «холодном» узле и на узле, где изменился только слой приложения размером 10 МБ?

14.2. Ответы

1. Только слой копирования исходных текстов и всё, что после него; слой зависимостей возьмётся из кэша.
2. Компилятор, заголовочные файлы, система сборки и кэш пакетов — всё, что нужно только при сборке.
3. Признак OOMKilled и фактическое потребление памяти под нагрузкой.
4. Память — жёсткий предел: превышение приводит к остановке ядром. Процессор — квота: процесс тормозится, но продолжает работать.
5. Значения переменных сохраняются в метаданных образа, а удалённый файл остаётся в предыдущем слое.
6. Данные исчезнут вместе с контейнером: сохранять нужно в томе или внешнем хранилище.
7. Изоляция не абсолютна: уязвимость в ядре или неверная настройка даёт больше возможностей процессу, работающему от суперпользователя.
8. Инфраструктура узнаёт, готов ли контейнер обслуживать запросы, и может заменить неисправный.
9. Он изменяем: разные узлы могут получить разные версии, а откат невозможен.
10. 8 секунд и 0,2 секунды соответственно.

15. Типичные ошибки и контрольные вопросы

15.1. Типичные ошибки

Ошибка	Признак и исправление
Демон Docker не запущен	`Cannot connect to the Docker daemon`; запустить Docker Desktop или `sudo systemctl start docker`
Команды выполняются в PowerShell вместо WSL	различия в путях и кавычках; открыть терминал Ubuntu
Пользователь не в группе docker	`permission denied ... docker.sock`; добавить в группу
Копирование вне контекста сборки	`COPY failed: forbidden path`; перенести файлы в контекст
Установка и очистка в разных RUN	образ не уменьшается; объединить в одну инструкцию
Часто меняющийся COPY выше установки пакетов	кэш сбрасывается, сборка идёт минуты вместо секунд
Запись в каталог без прав	Permission denied; см. пункт 6.2
Тег latest в описании развёртывания	непредсказуемая версия; указывать версию или дайджест
Образ другой архитектуры	`exec format error`; собирать образ под архитектуру узла

15.2. Контрольные вопросы

1. Чем контейнер отличается от виртуальной машины? Какой ресурс у контейнеров общий?
2. Какие пространства имён ядра Linux используются для изоляции контейнера и что именно изолирует каждое?
3. Что ограничивают контрольные группы? Приведите команды Docker, задающие пределы процессора и памяти.
4. Почему в контейнере нельзя запустить ядро другой операционной системы?
5. Что такое образ и что такое слой образа? Какие слои доступны для записи?
6. Сколько места на диске займут десять контейнеров, запущенных из одного образа размером 140 МБ?
7. Чем отличается ENTRYPOINT от CMD? Что произойдёт, если заданы оба?
8. Что такое контекст сборки? Зачем нужен файл .dockerignore?
9. Как построен кэш сборки? Почему инструкции, которые меняются редко, ставят в Dockerfile выше?
10. Что такое многоэтапная сборка и какой выигрыш она даёт для программы на компилируемом языке?
11. Почему образ на основе distroless меньше образа на основе ubuntu и что теряется при таком выборе?
12. Зачем в образе создают непривилегированного пользователя? Какую команду использует Dockerfile?
13. Чем отличается монтирование каталога хоста (bind mount) от именованного тома (volume)?
14. Почему запись в смонтированный каталог может завершиться ошибкой Permission denied и как это исправить?
15. Что такое тег образа и чем он отличается от дайджеста (digest)? Почему тег latest не годится для промышленной эксплуатации?
16. Что такое реестр образов? Назовите три реестра и укажите, чем Amazon ECR отличается от Docker Hub.
17. Что такое проверка состояния контейнера (HEALTHCHECK) и кто её использует?
18. Какие сервисы AWS запускают контейнеры? Чем AWS Fargate отличается от Amazon ECS на инстансах EC2?
19. Почему секреты нельзя передавать инструкцией ENV или копировать в образ на промежуточном слое?
20. Как узнать, из каких слоёв состоит образ и какой слой занимает больше всего места?

21. Чем тег образа отличается от дайджеста? Почему в описании развёртывания ссылаются на дайджест или неизменяемый тег, а не на latest?
22. Что именно хешируется при вычислении дайджеста образа?
23. Зачем реестр образов размещают в том же регионе, что и вычислительные ресурсы? Назовите три причины.
24. Чем сканирование при загрузке отличается от постоянного сканирования? Почему образ, безопасный в марте, может стать уязвимым в апреле без единого изменения?
25. Зачем реестру политика хранения и как сформулировать её так, чтобы нельзя было удалить работающую версию?
26. Что описывает определение задачи и чем оно отличается от задачи и от службы?
27. Что такое ревизия определения задачи и как через неё выполняются обновление и откат?
28. Можно ли заказать задачу с 0,25 vCPU и 8 ГБ памяти? Ответ обоснуйте таблицей допустимых сочетаний.
29. Почему при запуске без управления серверами каждой задаче нужен маршрут к реестру образов?
30. Как размер образа влияет на время запуска задачи и какой приём сборки помогает его сократить?
31. Чем роль исполнения отличается от роли задачи? Какие права выдаются каждой из них?
32. Почему секрет, однажды попавший в слой образа, считается скомпрометированным даже после удаления файла в следующем слое?
33. Что произойдёт при обновлении службы, если новая ревизия не проходит проверку состояния? Почему это правильное поведение?
34. Сформулируйте условие, при котором собственные инстансы выгоднее запуска без управления серверами.

15.3. Ошибки, которые проявляются только в облаке

Ошибка	Как проявляется	Как исправить
Образ собран под одну архитектуру	`exec format error` при запуске	многоархитектурная сборка или явная платформа
Ссылка на тег latest	разные версии на разных узлах, откат невозможен	неизменяемый тег или дайджест
Секрет передан через ENV при сборке	значение видно в метаданных образа	секрет подставляется при запуске
Приложение не обрабатывает сигнал	обрыв запросов при обновлении службы	приложение — первый процесс, обработка завершения
Состояние внутри контейнера	данные теряются при перезапуске задачи	внешнее хранилище
Проверка состояния всегда «готов»	запросы уходят в неисправную задачу	проверка отражает способность обслуживать
Предел памяти назначен «на глаз»	остановка ядром под нагрузкой	измерение потребления, затем предел с запасом
Нет маршрута к реестру из подсети	задача не запускается: образ недоступен	шлюз или конечная точка сервиса

16. Лабораторная работа № 5

Упаковка приложения в контейнер

16.1. Цель и порядок выполнения

Цель — упаковать программу, собираемую CMake, в контейнер тремя способами, сравнить результаты, научиться управлять кэшем сборки, ресурсами и сохранением данных, а также реализовать индивидуальное расширение программы и продемонстрировать его работу в контейнере (задание 7 — самостоятельной реализации индикатора).

Опорные файлы: каталог `quantflow` (библиотека `quant_core`, программа `quant-cli`, тесты и три варианта Dockerfile) и каталог `services/risk-model` внутри проекта (сервис на Python). Опорные файлы всех последующих занятий распаковываются в корень этого же проекта: он развивается от занятия к занятию и к концу курса содержит все службы, описания развёртывания и конвейер. Библиотека `quant_core` — развитие библиотеки, созданной на занятии 2: к функциям статистики добавлены индикаторы и детерминированный генератор котировок.

16.2. Требования к отчёту

1. Отчёт PDF: номер варианта, команды и их вывод по каждому заданию.
2. Таблица размеров образов и времени сборки (задания 2–4).
3. Исходный текст вашей функции индикатора, теста и изменённого Dockerfile (задание 7).
4. Файл схемы .drawio и изображение PNG (задание 6).
5. Выводы: не менее трёх утверждений.

16.3. Таблица индивидуальных параметров

Таблица 16.1 — Параметры вариантов

№	Инструмент	Зерно	Начальная цена, копеек	Шаг, копеек	Интервал, мс	Пределы --cpus / --memory	Индикатор для задания 7
1	ALFA	1037	5750	10	500	1 / 512m	Bollinger %B
2	BRAVO	1074	6500	8	1000	2 / 128m	MACD
3	CHARLIE	1111	7250	6	250	0,5 / 256m	Скорость изменения ROC
4	DELTA	1148	8000	4	500	1 / 512m	Моментум
5	ECHO	1185	8750	11	1000	2 / 128m	Z-оценка цены
6	FOXTROT	1222	9500	9	250	0,5 / 256m	Коэффициент вариации
7	GOLF	1259	10250	7	500	1 / 512m	Отношение доходности к риску
8	HOTEL	1296	11000	5	1000	2 / 128m	Нижнее полуотклонение
9	INDIA	1333	11750	3	250	0,5 / 256m	Медиана окна
10	JULIETT	1370	12500	10	500	1 / 512m	Взвешенная скользящая средняя WMA
11	KILO	1407	13250	8	1000	2 / 128m	Число смен направления
12	LIMA	1444	14000	6	250	0,5 / 256m	Длиннейшая серия роста
13	MIKE	1481	14750	4	500	1 / 512m	Асимметрия лог-доходностей
14	NOVEMBER	1518	15500	11	1000	2 / 128m	Эксцесс лог-доходностей
15	OSCAR	1555	16250	9	250	0,5 / 256m	Наибольшая длительность просадки

16.4. Задания 1–6

Задание 1. Установка и проверка

Установите Docker по разделу 2 и выполните проверку. В отчёт включите вывод `docker version` (версии клиента и сервера), строки `Operating System`, `Architecture`, `Storage Driver` из `docker info` и результат `docker run --rm hello-world`. Ответьте: какая операционная система указана в `docker info` и почему она может отличаться от вашей.

Задание 2. Три способа сборки

Соберите три образа из опорного проекта и сравните их:

```
cd quantflow
docker build -t quant-cli:single      -f docker/Dockerfile.single .
docker build -t quant-cli:multi       -f docker/Dockerfile.multi .
docker build -t quant-cli:distroless  -f docker/Dockerfile.distroless .
docker images quant-cli --format '{{.Tag}}\t{{.Size}}'
```

Запустите каждый образ с параметрами своего варианта и убедитесь, что вывод одинаков:

```
docker run --rm quant-cli:multi --symbol ИНСТРУМЕНТ --seed ЗЕРНО --base ЦЕНА \
    --step ШАГ --interval ИНТЕРВАЛ --count 1000 --sma N --ema N --rsi N
```

В отчёте: таблица «способ сборки — размер образа», вывод программы, ответ на вопрос, почему различаются размеры при одинаковом поведении программы.

Задание 3. Слои образа

Выполните `docker history quant-cli:multi` и `docker history quant-cli:single`. Определите: какой слой самый большой в каждом образе и какой инструкцией он создан. Ответьте, почему слой с исполняемым файлом занимает около 100 КБ, а образ — сотни мегабайт.

Задание 4. Кэш сборки

1. Пересоберите образ `quant-cli:multi` без изменений и запишите время: все шаги должны быть взяты из кэша (`CACHED`).
2. Измените файл `apps/quant-cli/main.cpp` так, чтобы программа печатала строку с номером вашего варианта, и пересоберите образ с ключом `--progress=plain`. Запишите, какие шаги выполнены заново, а какие взяты из кэша, и время сборки.
3. Переставьте в `docker/Dockerfile.multi` инструкцию `COPY . .` выше инструкции `RUN apt-get ...`, пересоберите и сравните время. Верните исходный порядок.

В отчёте: таблица «действие — время сборки — какие шаги пересобраны» и объяснение наблюдаемого.

Задание 5. Запуск: ресурсы, пользователь, данные

1. Запустите контейнер с пределами своего варианта и выведите значения из файловой системы cgroup: `docker run --rm --cpus C --memory Mm --entrypoint sh quant-cli:multi -c 'cat /sys/fs/cgroup/cpu.max /sys/fs/cgroup/memory.max; id'`. Сопоставьте выведенные числа с заданными пределами.
2. Сохраните котировки в файл внутри именованного тома: создайте том `docker volume create var_NN`, запустите `docker run --rm -v var_NN:/data quant-cli:multi ... --csv /data/ticks.csv`, затем выведите первые строки файла из тома.
3. Повторите запись в каталог хоста (`-v "$PWD/out":/data`). Если появится ошибка прав, устраните её одним из способов пункта 6.2 и опишите, какой способ выбран и почему.

Задание 6. Сервис на Python и схема

1. Соберите образ сервиса из каталога `services/risk-model` (`docker build -t risk-model:1.0 services/risk-model`) и запустите его: `docker run -d --name risk -p 8000:8000 risk-model:1.0`. Проверьте `curl localhost:8000/health` и `curl localhost:8000/model-info`. Остановите контейнер.
2. Сравните Dockerfile сервиса на Python и Dockerfile программы на C++: укажите, зачем в первом файл `requirements.txt` копируется отдельно и раньше исходных текстов.
3. Постройте в draw.io схему: рабочее место, Amazon ECR, кластер ECS с задачей Fargate в частной подсети региона вашего варианта и CloudWatch Logs. Подпишите операции push и pull.

16.5. Задание 7. Собственный индикатор и его демонстрация в контейнере

Реализуйте индикатор своего варианта в опорной библиотеке, покройте его тестом, включите в программу командной строки и продемонстрируйте работу в контейнере. Файлы ниже указаны для опорного проекта на C++; при работе на другом языке им соответствуют файлы объявления, реализации и тестов вашего проекта.

1. Добавьте в `core/include/quant/indicators.hpp` объявление функции, а в `core/src/indicators.cpp` — её реализацию строго по формуле из таблицы 16.2.
2. Добавьте в `tests/test_core.cpp` не менее двух проверок: на коротком ряду, значение для которого вычислено вручную, и на граничном случае (слишком короткий ряд — ожидается исключение).
3. Добавьте в программу командной строки вывод строки indicator <значение> с шестью знаками после запятой.
4. Соберите образ многоэтапной сборкой (тесты выполняются на этапе сборки: если тест не пройден, образ не создаётся) и запустите контейнер с параметрами своего варианта и `--count 1000`.
5. Продемонстрируйте преподавателю: неудачный тест останавливает сборку образа, а работающий контейнер печатает значение индикатора.

Таблица 16.2 — Индивидуальные индикаторы (p — ряд цен, T — индекс последней цены)

№	Индикатор	Определение
1	Bollinger %B	n = 20, k = 2: m и s — среднее и выборочное СКО последних 20 цен; %B = (p_T − (m − 2s)) / (4s)
2	MACD	$EMA_{12}(T) - EMA_{26}(T)$; EMA вычисляется функцией quant::ema
3	Скорость изменения ROC	ROC = (p_T / p_(T−n) − 1) × 100 %, n = 8
4	Моментум	M = p_T − p_(T−n), n = 20
5	Z-оценка цены	z = (p_T − m) / s, m и s — среднее и выборочное СКО последних 50 цен
6	Коэффициент вариации	CV = s / m по последним 100 ценам (s — выборочное СКО)
7	Отношение доходности к риску	mean(r) / s(r) по всем лог-доходностям (s — выборочное СКО)
8	Нижнее полуотклонение	sqrt((1/K) Σ $\min(r_i, 0)^2$), K — число лог-доходностей
9	Медиана окна	медиана последних 21 цены
10	Взвешенная скользящая средняя WMA	n = 10, веса 1, 2, …, 10 (наибольший — у последней цены)
11	Число смен направления	число индексов $i \geq 2$, для которых $\Delta_i \cdot \Delta_{i-1} < 0$, где $\Delta_i = p_i - p_{i-1}$
12	Длиннейшая серия роста	наибольшее число подряд идущих $\Delta_i > 0$
13	Асимметрия лог-доходностей	$g_1 = m_3 / m_2$^(3/2), m_k = (1/K) Σ $(r_i - \bar{r})$^k
14	Эксцесс лог-доходностей	$g_2 = m_4 / m_2^2 - 3$, m_k = (1/K) Σ $(r_i - \bar{r})$^k

№	Индикатор	Определение
15	Наибольшая длительность просадки	наибольшее число подряд идущих котировок, строго ниже максимума всех предшествующих цен

Обратите внимание. Во всех определениях выборочное СКО вычисляется с делением на (n − 1), центральные моменты m_k — с делением на число значений. Лог-доходности: r_i = ln(p_i / p_{i−1}). Значение индикатора вычисляется по ряду из 1000 котировок, построенному с параметрами вашего варианта.

Задание 8. Реестр образов: теги и дайджест

1. Присвойте собранному образу два тега: `quantflow/quant-cli:1.0.0` и `quantflow/quant-cli:latest`; приведите вывод `docker images` и объясните, почему размер занятого места не удвоился.
2. Получите дайджест образа и приведите его; объясните, что именно хешируется.
3. Измените одну строку в исходном тексте, пересоберите образ под тем же тегом `latest`, снова получите дайджест и покажите, что тег теперь указывает на другой образ.
4. Сохраните образ в файл (`docker save`) и приведите размер файла; сравните его с размером образа в выводе `docker images` и объясните различие.
5. Ответьте: почему в описании развёртывания ссылаются на дайджест или на неизменяемый тег версии, а не на `latest`; какое правило политики хранения вы задали бы для своего репозитория и почему.

Задание 9. Размер задачи и способ запуска

По измерениям задания 5 (пределы ресурсов) определите потребление вашего контейнера и выполните расчёты:

1. выберите наименьшее допустимое сочетание процессора и памяти задачи из таблицы раздела 8, в которое помещается ваш контейнер с запасом не менее 30 % по памяти; приведите расчёт;
2. укажите, сколько задач такого размера потребуется при трёх репликах и во что это обойдётся по сравнению с одним инстансом, способным вместить все три реплики (сравнение по ресурсам, не по валюте);
3. ответьте: при каком профиле нагрузки выгоднее запуск без управления серверами, а при каком — собственные инстансы; приведите условие в виде неравенства по доле времени работы;
4. перечислите, какие права нужны роли исполнения и какие — роли задачи для вашего сервиса.

16.6. Задания повышенной сложности

Задания этого раздела выполняются по желанию и оцениваются дополнительно. Они не требуют новых средств: всё необходимое уже установлено и разобрано. Требуется другое — самостоятельно спланировать измерение или расчёт, выбрать способ проверки и объяснить полученный результат. Задание засчитывается только с приведённой методикой: что измерялось, чем, сколько раз и почему полученному числу можно верить.

Задание П1. Кэш сборки в числах. Измерьте время повторной сборки образа при пяти видах изменений: правка исходного текста, правка файла зависимостей, добавление нового файла, изменение параметра сборки, изменение базового образа. Постройте таблицу «что изменилось —

какие слои пересобраны — время». Затем измените порядок инструкций так, чтобы самое частое изменение затрагивало наименьшее число слоёв, и повторите измерения.

Задание П2. Предел памяти и поведение приложения. Запустите расчёт своего варианта с пределами памяти 64, 96, 128 и 256 МиБ. Для каждого случая зафиксируйте: завершился ли контейнер, код завершения, признак остановки ядром, фактическое потребление. Определите наименьший предел, при котором расчёт устойчиво завершается, и объясните, почему брать именно это значение в промышленной среде нельзя.

Задание П3. Два способа уменьшить образ и их цена. Сравните три способа уменьшения образа: многоэтапная сборка, минимальный базовый образ, статическая сборка приложения. Для каждого приведите размер образа, время сборки, возможность диагностики внутри контейнера и ограничения. Сделайте вывод, какой способ вы выбрали бы для промышленной среды и какой — для учебной, с обоснованием.

Порядок оценивания. Каждое задание оценивается отдельно. Условие зачёта одинаково: приведены методика, результаты и объяснение. Результат без объяснения, как и объяснение без результата, не засчитывается.

16.7. Критерии оценивания

Составляющая	Что оценивается	Доля
Задания 1–3	три образа собраны, размеры и слои объяснены	18 %
Задание 4	эксперимент с кэшем, таблица времён, объяснение	14 %
Задание 5	пределы ресурсов, работа с томом, решённая задача прав	14 %
Задание 6	сервис запущен, различие Dockerfile объяснено, схема построена	19 %
Задание 7	корректная реализация индикатора, тесты, демонстрация в контейнере	20 %
Задание 8	реестр: теги и дайджест, объяснение неизменяемости	7 %
Задание 9	расчёт размера задачи и способа запуска с обоснованием	8 %

Важно. Задание 7 не принимается, если тест не проверяет вычисленное вручную значение или если образ собран без выполнения тестов.

16.8. Этап сквозного проекта: ядро проекта в контейнере

Этап переводит расчётное ядро проекта в контейнер. После него любой расчёт запускается одной командой на любой машине, а размер и содержимое образа становятся измеримыми величинами, которые войдут в модель стоимости.

Что сделать.

1. Перенесите библиотеку расчётов и программу командной строки в каталог проекта: core и apps/quant-cli.
2. Соберите образ quantflow/quant-cli:varNN многоэтапной сборкой с непривилегированным пользователем и проверкой состояния. Тег содержит номер варианта.
3. Включите в библиотеку собственный индикатор из задания 7 вместе с тестом, который проверяет вычисленное вручную значение.
4. Запустите расчёт в контейнере с параметрами своего варианта и сверьте sma, ema и rsi с таблицей Г.4 приложения Г к занятию 4.
5. Дополните схему проекта: реестр образов и задача, запускающая расчёт.

6. Внесите в журнал решений запись о выборе базового образа: размер, время запуска, что потеряно и что выиграно.

Что появляется в проекте: docker/Dockerfile.quant-cli и собранный образ; исходный текст собственного индикатора и теста; обновлённая diagrams/project.drawio; новая запись в docs/decisions.md.

Как проверяется на приёме:

— образ собирается из каталога проекта одной командой;

— в образе непривилегированный пользователь и проверка состояния;

— значения индикаторов совпадают с таблицей Г.4;

— собственный индикатор покрыт тестом с ручной проверкой;

— в слоях образа нет ключей, паролей и файлов вариантов других обучающихся.

Время и оценка. Ориентировочное время выполнения этапа — 40 минут. Этап предъявляется вместе с отчётом по лабораторной работе и оценивается отдельно от неё: он входит в накопительную составляющую отметки и в итоговую защиту сквозного проекта.

17. Подготовка к итоговой аттестации

Ниже приведены вопросы итоговой аттестации, которые опираются на материал этого занятия, и план ответа на каждый: перечень того, что должно прозвучать. План — не текст ответа, а проверочный список: если какой-то пункт не удаётся раскрыть своими словами, к нему стоит вернуться в тексте занятия.

17.1. Виртуальная машина и контейнер: сходство и различия. Пространства имён и контрольные группы

План ответа:

— что именно изолируется в каждом случае: оборудование или представление системы;

— пространства имён: какие бывают и что каждое скрывает;

— контрольные группы: ограничение и учёт ресурсов;

— общее ядро — источник и преимуществ (скорость запуска), и ограничений (изоляция слабее);

— когда нужна виртуальная машина, а когда достаточно контейнера.

17.2. Docker: образы, слои, многоэтапная сборка. Создание контейнера с приложением

План ответа:

— образ как набор слоёв; слой только для записи у контейнера;

— инструкции файла сборки и соответствие «инструкция — слой»;

— кэш слоёв и правило порядка: редко меняющееся ниже, часто меняющееся выше;

— многоэтапная сборка: что остаётся в итоговом образе и почему размер падает в разы;

— минимальный базовый образ: выигрыш и цена (отсутствие оболочки);

— непривилегированный пользователь, проверка состояния, неизменяемый тег или дайджест.

17.3. Запуск контейнеров в облаке: определение задачи, служба, запуск без управления серверами

План ответа:

— определение задачи: образ, ресурсы, порты, переменные, роль, проверка состояния;

— задача и служба: разовое выполнение против поддержания заданного числа копий;

— две роли: роль исполнения и роль задачи — что может каждая;

— запуск без узлов: что берёт на себя поставщик и как это отражается в оплате;

— допустимые сочетания процессора и памяти и правило округления вверх;

— выбор между своими узлами и запуском без узлов по характеру нагрузки.

18. Приложения

Приложение А. Чек-лист перед сдачей

Проверено	Что именно
☐	собраны три варианта образа, размеры приведены в таблице;
☐	показано влияние порядка инструкций на время повторной сборки;
☐	приведены слои образа и объяснено, что в каждом из них;
☐	выполнены запуски с ограничением памяти и процессора, объяснены коды завершения;
☐	показана работа с томом и решена задача прав доступа;
☐	образ собран многоэтапной сборкой, работает от непривилегированного пользователя и содержит проверку состояния;
☐	приведены теги и дайджест образа, объяснена неизменяемость;
☐	выбран размер задачи по измеренному потреблению с обоснованием запаса.

Приложение Б. Команды для реестра и оркестратора

Команда	Назначение
aws ecr create-repository --repository-name имя	создать репозиторий образов
aws ecr get-login-password | docker login --username AWS --password-stdin АДРЕС	вход в реестр
docker tag образ АДРЕС/имя:1.0.0	присвоить имя для реестра
docker push АДРЕС/имя:1.0.0	загрузить образ
aws ecr describe-images --repository-name имя	перечень образов и дайджестов
aws ecr describe-image-scan-findings --repository-name имя --image-id imageTag=1.0.0	находки сканирования
aws ecr put-lifecycle-policy --repository-name имя --lifecycle-policy-text file://policy.json	политика хранения
aws ecs register-task-definition --cli-input-json file://task.json	зарегистрировать определение задачи
aws ecs create-service --cluster K --service-name C --task-definition T:1 --desired-count 2	создать службу
aws ecs update-service --cluster K --service C --task-definition T:2	обновить службу на новую ревизию
aws ecs describe-services --cluster K --services C	состояние службы и число задач
aws logs tail /ecs/имя --follow	журналы задач

Команды приведены для полноты картины: в лабораторной работе они выполняются против эмулятора и часть из них поддерживается им лишь частично. Промышленное развёртывание описывается не командами, а конфигурацией (занятие 14).

Приложение В. Сводка команд

Команда	Назначение
`docker build -t имя:тег -f Dockerfile .`	собрать образ
`docker images`, `docker history`	список образов, слои образа
`docker run [параметры] образ [аргументы]`	запустить контейнер
`docker ps -a`, `docker logs`, `docker exec -it ... sh`	состояние, журнал, оболочка
`docker stop`, `docker rm`, `docker rmi`	остановить, удалить контейнер, удалить образ
`docker volume create|ls|rm`	работа с именованными томами
`docker system df`, `docker system prune`	занятое место и очистка

Приложение Г. Глоссарий

Термин	Значение
Образ	неизменяемый набор слоёв и метаданных, из которого создаётся контейнер
Слой	результат одной инструкции Dockerfile, изменившей файлы
Контекст сборки	каталог, передаваемый сборщику; из него копирует COPY
Многоэтапная сборка	несколько этапов FROM; в итог переносится только результат
distroless	образ без оболочки и пакетного менеджера, содержащий только программу
Том	хранилище данных вне слоя контейнера
Реестр	хранилище образов (Docker Hub, ghcr.io, Amazon ECR)
Дайджест	хеш содержимого образа: sha256:…
OCI	стандарт формата образа и среды выполнения
Fargate	режим запуска контейнеров AWS без управления серверами